Linux-based Programming Training

```
[sedu50@login03 Linux]$ ps -f
UID        PID   PPID  C STIME TTY          TIME CMD
sedu50   54536  54531  0 17:26 pts/3    00:00:00 -bash
sedu50   60903  54536  0 17:36 pts/3    00:00:00 ps -f
[sedu50@login03 Linux]$ echo $TERM
xterm-256color
[sedu50@login03 Linux]$ echo $SHELL
/bin/bash
```

```
#!/bin/bash
a=0

until [ ! $a -lt 10 ]
do
   echo $a
   a=`expr $a + 1`
done
```

April 07, 2026.

Tae Ho Yoon

KISTI

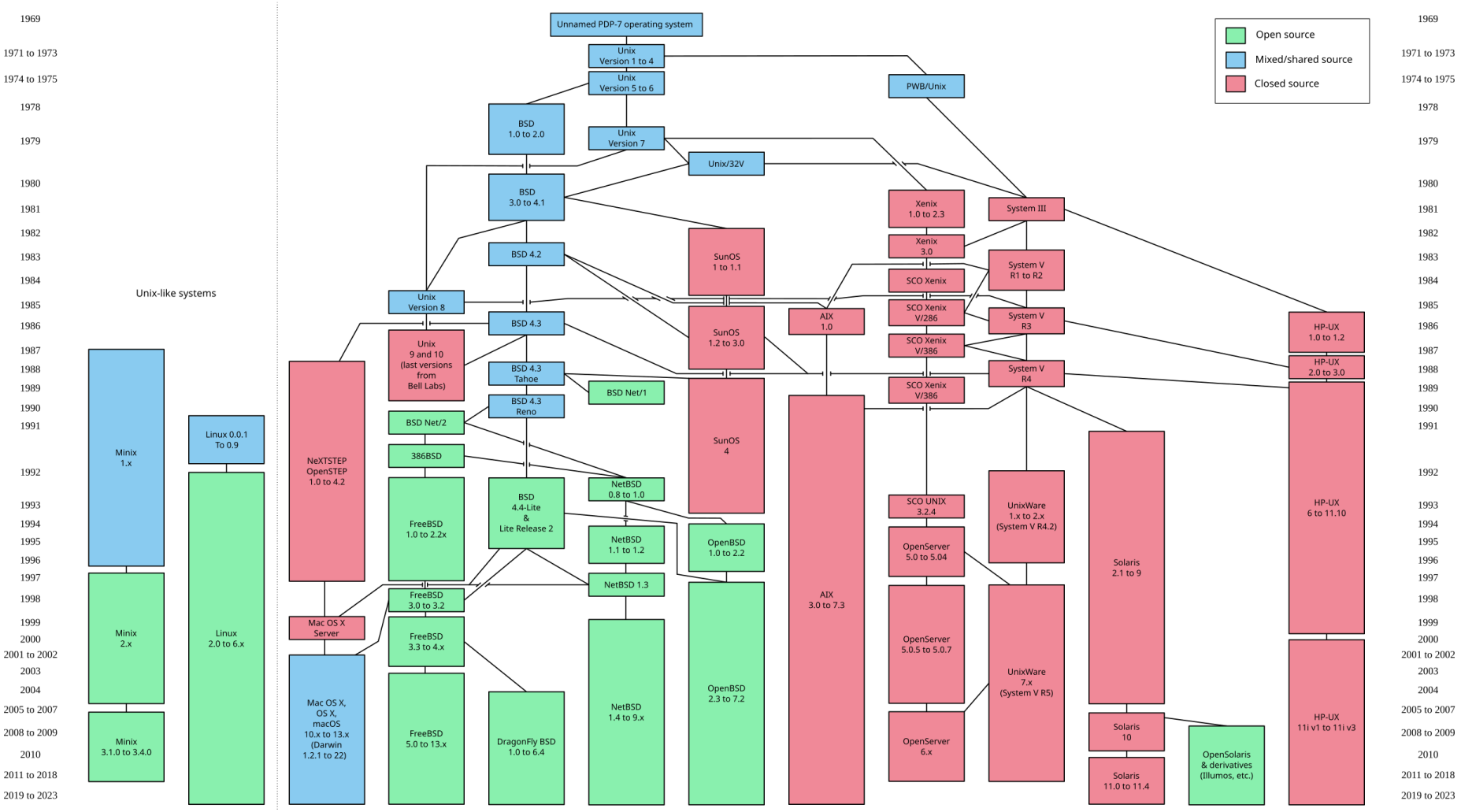
Agenda

- 14:00 ~ 14:40 Linux Overview and Usages
- 14:40 ~ 15:10 vi Editor, sed, and awk Command
- 15:10 ~ 15:30 Break
- 15:30 ~ 16:30 Shell Script Programming
- 16:30 ~ 17:00 Programming Understanding

Agenda

- 14:00 ~ 14:40 Linux Overview and Usages
- 14:40 ~ 15:10 vi Editor, sed, and awk Command
- 15:10 ~ 15:30 Break
- 15:30 ~ 16:30 Shell Script Programming
- 16:30 ~ 17:00 Programming Understanding

History of Unix/Linux



Reference: https://en.wikipedia.org/wiki/History_of_Unix

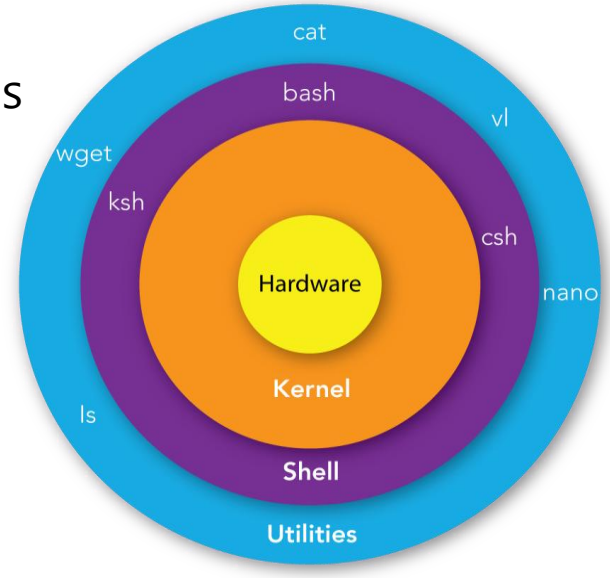
Structure of Linux

- Kernel
  - The core of the operating system
  - Management of system resources
- Shell
  - Command Interpreter
  - Passing commands between users and systems
- Terminal
  - An interface where you can enter commands
- File System
  - Has a hierarchical structure
  - A place to store information

Linux Kernel Version

ex) 3.6.35

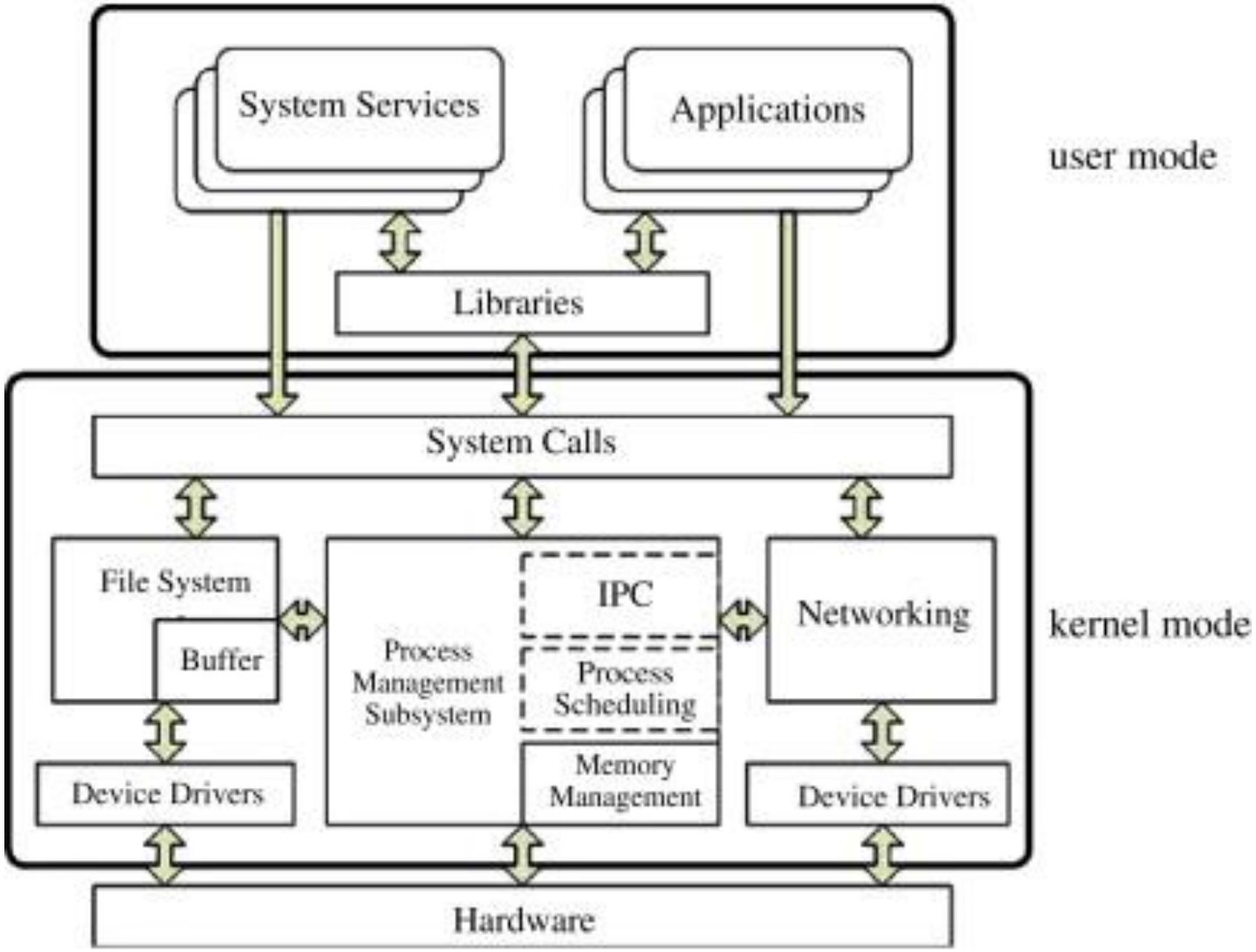
major number | minor number | revision number



❖ Linux is designed to allow multiple users to share a single system

Image Source: https://medium.com/@innovationchef/shell-scripting-i-linux-basics-22e1ba2a0437

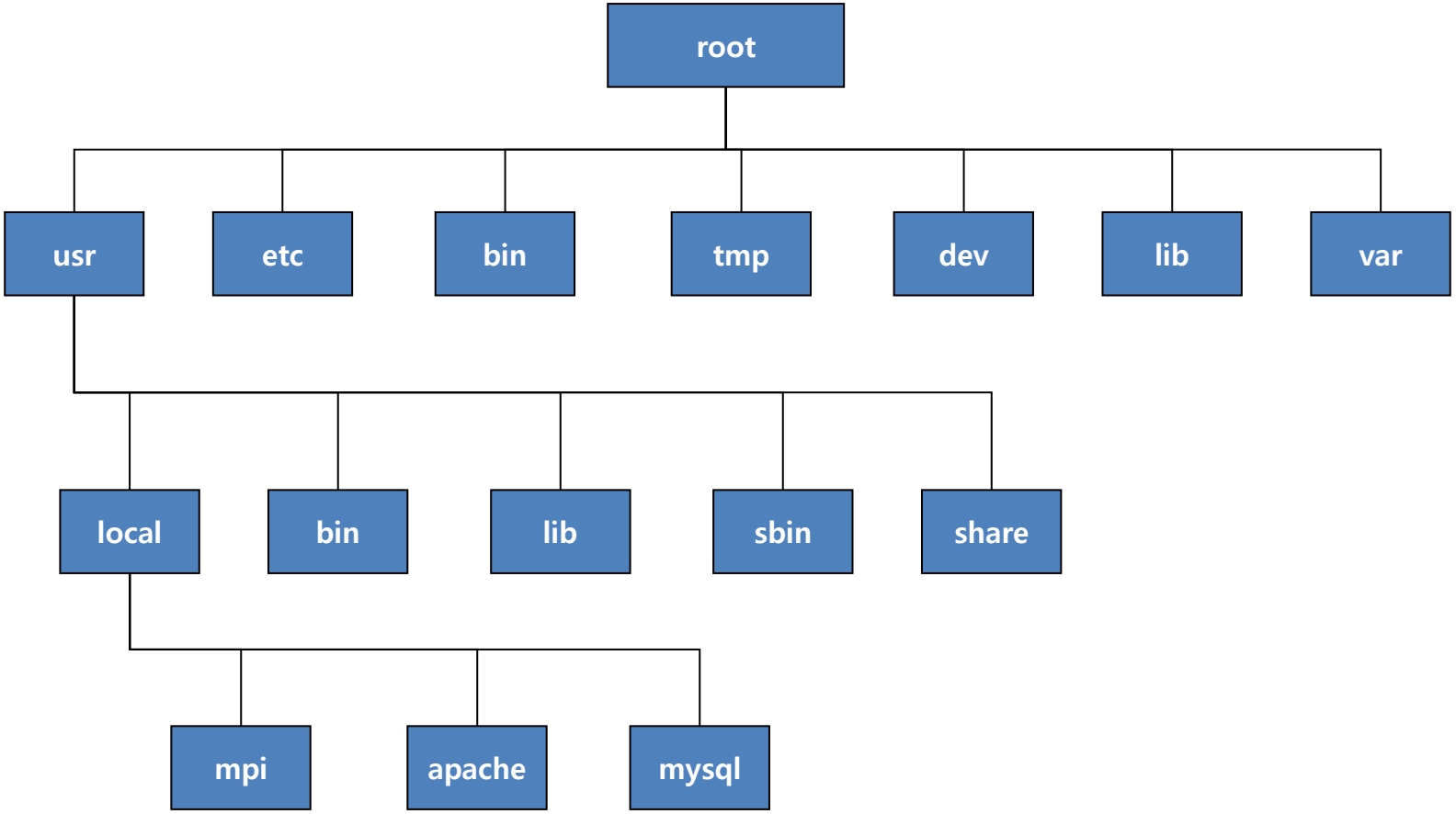
Structure of Linux



https://www.sciencedirect.com/science/article/pii/S0895717709003409#bb6

Structure of Linux

➢ File Hierarchy



File Hierarchy

Directory	Function
/usr	Contains shareable files
/etc	Management files used by the system (init, getty, mknod, motd, passwd, …)
/bin	Have a basic executable command file (cat, cp, date, echo, mv, pwd, …)
/tmp	Temporary directory
/dev	Directory containing device files Contains all input/output files of the system
/lib	Directory containing the basic program modules.
/var	Logs generated during system operation

Command Execution Structure

➢ Command Combinations

Command + Options + Arguments

➢ Command Methods

- command
  - Generally lowercase
  - What to do?

```
$ ls
```

- options
  - Improved command output
  - Use in combination with one or more other options.

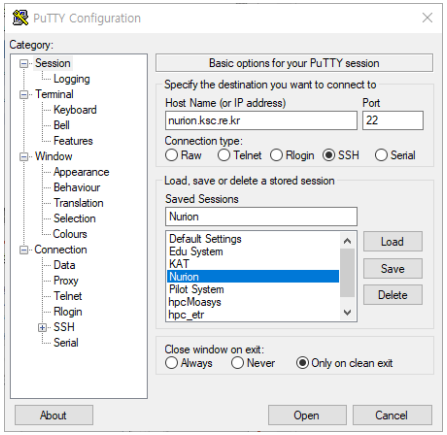
```
$ ls -al
```

- arguments
  - Execute according to user's command
  - Can have one or more arguments

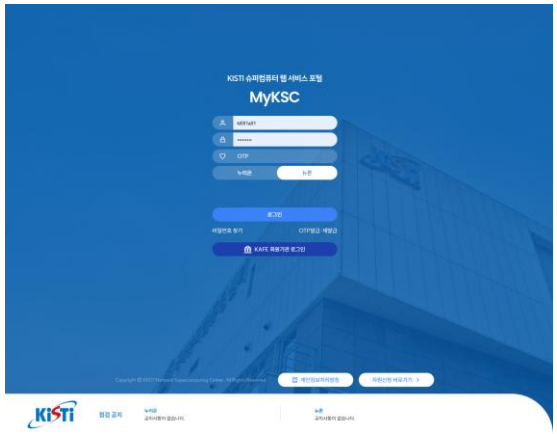
```
$ ls -alh /home01/sedu50
```

Linux System Connection Tools

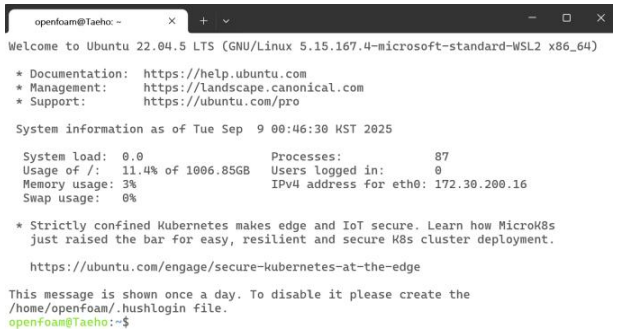
- Putty (https://www.putty.org/)



- MyKSC (https://portal.my.ksc.re.kr)
  - Web-based tool



- WSL2 (https://learn.microsoft.com/en-us/windows/wsl/install)

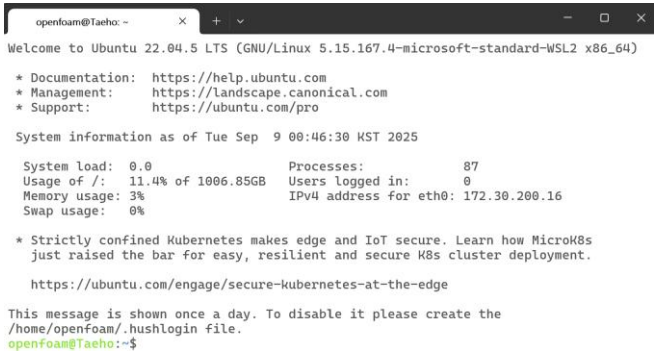


Linux in Window platform - WSL2

- Uninstalling WSL2
  - Type ‘power shell’ in the Windows search bar
  - $ wsl --version

    (If not version 2 then $ wsl --set-default-version 2)
  - $ wsl --update
  - Check Installed OS platform : $ wsl --list
  - $ wsl --unregister “Ubuntu name”
- Installing WSL2
  - $ wsl –install Ubuntu-22.04

  ※ Enter Microsoft Store in the search bar, then download and install Ubuntu 22.04.6 LTS or 18.04.6 LTS.



Terminal vs. Shell

- Terminal: An interface where you can enter commands

```
[ki001a01@login05 /]$ ls
applic  apps_ime  boot  dev  global  home    home_ime  lib64  media  opt   root  run   scratch      srv  tftpboot  usr
apps    bin       cm    etc  gpfs    home01  lib       local  mnt    proc  rpms  sbin  scratch_ime  sys  tmp       var
[ki001a01@login05 /]$
```

- Shell: Processes the entered command and outputs the results to the terminal
- GNOME Terminal

```
[sedu50@login03 Linux]$ ps -f
UID        PID   PPID  C STIME TTY          TIME CMD
sedu50   54536  54531  0 17:26 pts/3    00:00:00 -bash
sedu50   60903  54536  0 17:36 pts/3    00:00:00 ps -f
[sedu50@login03 Linux]$ echo $TERM
xterm-256color
[sedu50@login03 Linux]$ echo $SHELL
/bin/bash
```

Linux Environment Checking

- uname –r

```
3.10.0-1062.18.1.el7.x86_64
```

- hostnamectl

```
Static hostname: login05
        Icon name: computer-server
          Chassis: server
       Machine ID: e27c96ba49f34b0682d9f5282c6ba65f
          Boot ID: 0b86cae6d46d45208d4bee9da51c1844
 Operating System: Red Hat Enterprise Linux Atomic Host 7.4.0.4
      CPE OS Name: cpe:/o:redhat:enterprise_linux:7.4:GA:atomic-host
           Kernel: Linux 3.10.0-1062.18.1.el7.x86_64
     Architecture: x86-64
```

- cat /proc/version

```
Linux version 3.10.0-1062.18.1.el7.x86_64
(mockbuild@kbuilder.bsys.centos.org) (gcc version 4.8.5 20150623 (Red
Hat 4.8.5-39) (GCC) ) #1 SMP Tue Mar 17 23:49:17 UTC 2020
```

Linux Environment Checking

➢ cat ~/.bashrc

```
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi
.
.
.
```

※ /etc/profile, /etc/profile.d

Process

- The basic unit for allocating running programs or system resources(similar as ‘task’)
- Each process can have one parent process and many new child processes.
- Process states
  - create
  - running
  - ready
  - wait
  - quit
- All programs on Linux/Unix systems run as processes.

Process

- Check which processes are running on the system using the 'ps' command.
- Each process has a unique PID (process id)
- Foreground
  - The process that is usually performed is performed in the foreground.
  - One command is executed and waiting until execution is completed
- Background
  - Must specify that it should run in the background in order to perform multi-tasking → Add '&' to the end of the command

```
[sedu50@login02 Linux]$ ./pi.x
PI= 3.141592653589742 (Error = 5.151435e-14)
Elapsed Time = 65.442025, [sec]
[sedu50@login02 Linux]$ ./pi.x &
[1] 247884
[sedu50@login02 Linux]$ ps
   PID TTY          TIME CMD
203092 pts/93   00:00:00 bash
247884 pts/93   00:00:02 pi.x
247885 pts/93   00:00:00 ps
```

```
[sedu50@login02 Linux]$
PI= 3.141592653589742 (Error = 5.151435e-14)
Elapsed Time = 65.515562, [sec]

[1]+  Done                    ./pi.x
```

※ File download: wget https://github.com/auseyoon/OpenMP.git/pi.x .

Foreground & Background Process

```
[sedu50@login02 Linux]$ ls
pi.c  pi.x
[sedu50@login02 Linux]$ ./pi.x
-------------------------------------
```

foreground

```
^Z
[1]+  Stopped                 ./pi.x
```

suspend(ctrl + z)

```
[sedu50@login02 Linux]$ ./pi.x &
[2] 251745
```

background

```
[sedu50@login02 Linux]$ -------------------------------------

[sedu50@login02 Linux]$ jobs
[1]+  Stopped                 ./pi.x
[2]-  Running                 ./pi.x &

[sedu50@login02 Linux]$ bg 1
[1]+ ./pi.x &
[sedu50@login02 Linux]$ jobs
[1]-  Running                 ./pi.x &
[2]+  Running                 ./pi.x &
```

Resume suspended pi.x in the background

```
[sedu50@login02 Linux]$ fg 1
./pi.x
PI= 3.141592653589742 (Error = 5.151435e-14)
Elapsed Time = 66.226016, [sec]
----------------------------------------
PI= 3.141592653589742 (Error = 5.151435e-14)
Elapsed Time = 85.428716, [sec]
----------------------------------------
[2]   Done                    ./pi.x
```

Resume pi.x in foreground

System Monitoring

➢ top

$ top -d time(second) : Refresh time

Sytem
Total running time
(= $ uptime)

```
top - 11:44:05 up 24 days, 18:18,  122 users,  load average: 1.48, 1.57, 1.04
Tasks: 934 total,   2 running, 918 sleeping,  14 stopped,   0 zombie
%Cpu(s):  2.2 us,  4.4 sy,  0.0 ni, 93.3 id,  0.0 wa,  0.0 hi,  0.1 si,  0.0 st
KiB Mem : 39487516+total, 28259836+free, 16995640 used, 95281168
KiB Swap: 16777212 total, 16777212 free,        0 used. 3542777

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
247894 x1698a02  20   0  147968   4292   1248 S  24.1  0.0  16:47.35 rsync
273168 root      20   0  169124   7688   4828 S  23.4  0.0   1:18.78 sshd
294493 applic    20   0    4340    344    276 R   7.9  0.0   0:00.24 rm
273352 pmg96     20   0   74268   3516   2636 S   5.0  0.0   0:18.00 sftp-server
274996 applic    20   0  137256   3116   1540 S   4.3  0.0   0:06.19 configure
247659 root      20   0  173428  11888   4848 S   2.3  0.0   1:43.12 sshd
436389 root      20   0       0      0      0 S   1.3  0.0   1:17.02 kworker/2:2
   421 root     -51   0       0      0      0 S   1.0  0.0  10:35.21 irq/393-
hfi1_0
287636 sedu50    20   0  175120   3512   1904 R   1.0  0.0   0:02.00 top
 66993 root      20   0 1493084 216724  23684 S   0.7  0.1 155:17.84 klnagent
408474 root      20   0       0      0      0 S   0.7  0.0   0:24.33 kworker/1:2
     1 root      20   0  192528   5692   2520 S   0.3  0.0   9:02.27 systemd
```

CPU,
Memory
Usage Rate

System Monitoring

- top
  - CPU Status showing
    - '1' : Shows the status of the CPU
    - 'q' : Command Quit

```
top - 11:46:16 up 24 days, 18:20, 122 users,  load average: 1.68, 1.60, 1.12
Tasks: 934 total,   3 running, 917 sleeping,  14 stopped,   0 zombie
%Cpu0  :  4.0 us,  1.3 sy,  0.0 ni, 94.1 id,  0.0 wa,  0.0 hi,  0.7 si,  0.0 st
%Cpu1  :  1.3 us, 38.2 sy,  0.0 ni, 60.2 id,  0.0 wa,  0.0 hi,  0.3 si,  0.0 st
%Cpu2  :  1.0 us, 13.2 sy,  0.0 ni, 85.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
…
%Cpu23 :  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem : 39487516+total, 28352208+free, 17000044 used, 94353048 buff/cache
KiB Swap: 16777212 total, 16777212 free,        0 used. 35431958+avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
273168 root      20   0  169124   7688   4828 R  28.4  0.0   1:48.66 sshd
247894 x1698a02  20   0  147968   4292   1248 S  23.8  0.0  17:18.32 rsync
295946 applic    20   0    6580    224    172 R   5.9  0.0   0:00.18 sed
273352 pmg96     20   0   74268   3516   2636 S   4.3  0.0   0:22.48 sftp-server
    13 root      rt   0       0      0      0 S   3.0  0.0   1:33.20 migration/1
247659 root      20   0  173428  11888   4848 S   2.6  0.0   1:46.25 sshd
    12 root      rt   0       0      0      0 S   2.0  0.0   0:23.20 watchdog/1
```

Control key

Ctrl Key	stty	Function
Ctrl – C	intr	Stop current command
Ctrl – D	eof	End of input
Ctrl – ₩	quit	Stop current command if Ctrl-C doesn't work
Ctrl – S	stop	Stop screen output
Ctrl – Q		Resume screen output
Ctrl – ? or DEL	erase	Delete last character
Ctrl – U	kill	Delete entire command line
Ctrl – Z	susp	Suspend current command

Redirection

- The most powerful command line supported by Linux
  - Ability to redirect to a file as if displaying the results of a command or program on the screen.
  - Use files as input or output
  - The results of a command can be used as input to another command.
  - Send the results of the executable program to a file

    ex) $ test.exe > test.out

    $ cat test.out

    ex) $ cal > october.txt

    $ cat october.txt
  - Use the contents of a file as input to a command
    - test.x < input.dat > output.dat
    - mail mark < letter

```
$ cat test.f90
PROGRAM TEST
IMPLICIT NONE
INTEGER::I,J
READ *, I
J=10*I
PRINT*, J
END PROGRAM TEST
$ gfortran test.f90 -o test.x
$ ./test.x
5
          50
$ cat input.dat
5
$ ./test.x <input.dat >output.dat
$ cat output.dat
          50
```

Pipe

- Pipe '|' between commands
  - Allows the result of one command to become the input for another command.

Ex) $ who | grep "sedu"

```
[sedu50@login03 Linux]$ who | grep "sedu"
sedu50   pts/82       2019-01-24 11:22
sedu50   pts/84       2019-01-24 11:22
```

File Property

- ls -al

```
[sedu50@login03 Linux]$ ls -l
total 4
-rw-r----- 1 sedu50 in0163 1475 Jan 24 16:21 my.txt
```

permission

hard link number

user

group

size

create day/time

file name

Link Types

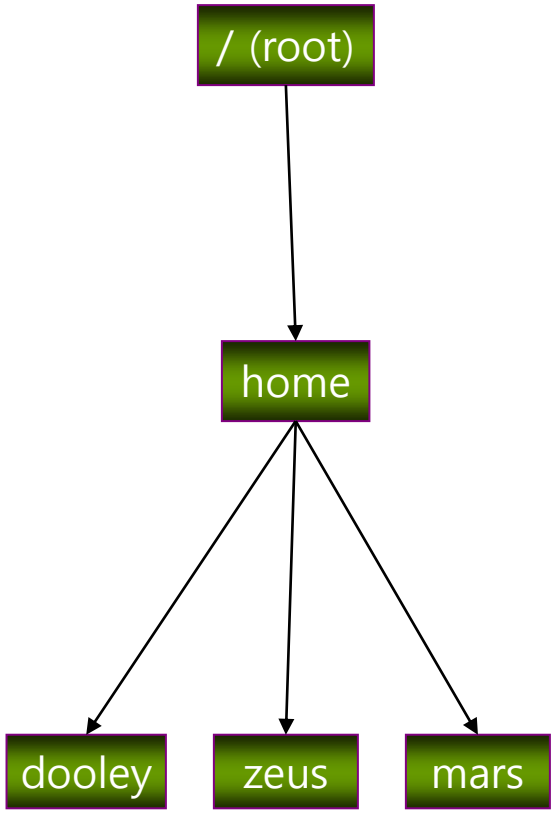
➢ Hard & Symbolic Link

- Hard Link : ln [link original file] [link file]
- Symbolic Link : ln -s [link original file] [link file]

```
[sedu50@login02 Linux]$ cat ori.txt
Original file
[sedu50@login02 Linux]$ ln ori.txt ori_hard.txt
[sedu50@login02 Linux]$ ln -s ori.txt ori_symbol.txt
[sedu50@login02 Linux]$ ls -lih
total 8.0K
144123145593350155 -rw-r--r-- 2 sedu50 in0163 14 Jan 25 10:04 ori_hard.txt
144123145593350159 lrwxrwxrwx 1 sedu50 in0163  7 Jan 25 10:05 ori_symbol.txt -> ori.txt
144123145593350155 -rw-r--r-- 2 sedu50 in0163 14 Jan 25 10:04 ori.txt
[sedu50@login02 Linux]$ cat ori.txt (ori_symbol.txt & ori_hard.txt)
Original file
[sedu50@login02 Linux]$ vi ori.txt
[sedu50@login02 Linux]$ cat ori.txt(ori_hard.txt & ori_symbol.txt)
Original file
Modified....
[sedu50@login02 Linux]$ rm ori.txt
[sedu50@login02 Linux]$ ls -lih
total 4.0K
144123145593350155 -rw-r--r-- 1 sedu50 in0163 27 Jan 25 10:06 ori_hard.txt
144123145593350159 lrwxrwxrwx 1 sedu50 in0163  7 Jan 25 10:05 ori_symbol.txt -> ori.txt
[sedu50@login02 Linux]$ cat ori_symbol.txt
cat: ori_symbol.txt: No such file or directory
```

How to specify a path

- Absolute Path
  - /(root directory) based location
- Relative Path
  - .(current directory) based location
- (e.g.) zeus -> mars
  - cd /home/mars
  - cd ../mars



Directory/File Management

- cat
  - Create a simple text file or check the contents of a text file

  ex) cat /etc/passwd

- echo
  - Print text on screen

```
[sedu50@login03 Linux]$ echo "hello"
hello
[sedu50@login03 Linux]$ echo $HOME
/home01/sedu50
[sedu50@login03 Linux]$ echo $PWD
/home01/sedu50/EDU/Linux
```

※ $export

➢ cat and echo

– After writing a text file with the cat command, use **Ctrl + c** to exit.

```
[sedu50@login03 Linux]$ cat >my1.txt
abc
def
^C
[sedu50@login03 Linux]$ cat > my2.txt
abcc
def
^C
[sedu50@login03 Linux]$ ls -l
total 8
-rw-r--r-- 1 sedu50 in0163 8 Jan 24 15:14 my1.txt
-rw-r--r-- 1 sedu50 in0163 9 Jan 24 15:14 my2.txt
[sedu50@login03 Linux]$ echo "Hello, World"
Hello, World
[sedu50@login03 Linux]$ cat my1.txt
abc
def
[sedu50@login03 Linux]$ cat my2.txt
abcc
def
```

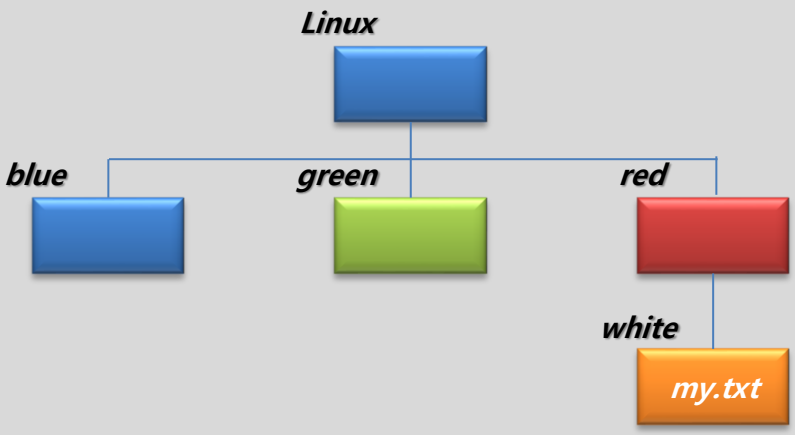
Directory/File Management

- cd
  - Directory move command
- pwd
  - Show current directory location
- mkdir
  - Used when creating a new directory
- rmdir
  - Used when deleting a directory
- touch
  - Change the modification date of a file if the file exists
  - Create 0 kb file if file does not exist
- cp
  - File copy command
  - Add -a option if you want to keep properties

Directory/File Management

➢ cd, pwd, mkdir, touch

```
[sedu50@login03 etc]$ cd
[sedu50@login03 ~]$ pwd
/home01/sedu50
[sedu50@login03 ~]$ cd EDU/
[sedu50@login03 EDU]$ ls
Linux
[sedu50@login03 EDU]$ cd Linux/
[sedu50@login03 Linux]$ mkdir red
[sedu50@login03 Linux]$ mkdir green
[sedu50@login03 Linux]$ mkdir blue
[sedu50@login03 Linux]$ ls
blue  green  red
[sedu50@login03 Linux]$ pwd
/home01/sedu50/EDU/Linux
[sedu50@login03 Linux]$ cd red
[sedu50@login03 red]$ pwd
/home01/sedu50/EDU/Linux/red
[sedu50@login03 red]$ mkdir white
[sedu50@login03 red]$ cd white/
[sedu50@login03 white]$ touch my.txt
[sedu50@login03 white]$ ls -l
total 0
-rw-r--r-- 1 sedu50 in0163 0 Jan 24 14:07 my.txt
```



Directory/File Management

➢ cp and clear

```
[sedu50@login03 white]$ clear
[sedu50@login03 white]$ cp my.txt ../../green/
[sedu50@login03 white]$ cd ../../green/
[sedu50@login03 green]$ pwd
/home01/sedu50/EDU/Linux/green
[sedu50@login03 green]$ ls
my.txt
[sedu50@login03 green]$ ls -l
total 0
-rw-r--r-- 1 sedu50 in0163 0 Jan 24 14:16 my.txt
```

Directory/File Management

- rm
  - Delete a file or directory(-rf option)
  - Option
    - -i : Confirm when deleting
    - -f : Force Delete
    - -r : When deleting a directory, all subdirectories and files are also deleted.
    - -v : Output of ‘rm’ command progress
- mv
  - Used when renaming files and directories or moving their paths.
  - When using the -b option : If there is an identical file, create a backup file.

Directory/File Management

➢ rm and rmdir

```
[sedu50@login03 red]$ cd red
[sedu50@login03 red]$ pwd
/home01/sedu50/EDU/Linux/red
[sedu50@login03 red]$ ls
white
[sedu50@login03 red]$ rmdir white/
rmdir: failed to remove ‘white/’: Directory not empty
[sedu50@login03 red]$ cd white/
[sedu50@login03 white]$ ls
my.txt
[sedu50@login03 white]$ rm my.txt
[sedu50@login03 white]$ cd ../
[sedu50@login03 red]$ rmdir white/
[sedu50@login03 red]$ ls
```

Directory/File Management

- rm –rf : Delete directory containing files

```
[sedu50@login03 red]$ mkdir white
[sedu50@login03 red]$ cd white/
[sedu50@login03 white]$ touch my.txt
[sedu50@login03 white]$ ls
my.txt
[sedu50@login03 white]$ cd ../
[sedu50@login03 red]$ ls
white
[sedu50@login03 red]$ rm -rf white/
[sedu50@login03 red]$ ls
```

Directory/File Management

➢ rm and mv

```
[sedu50@login03 Linux]$ mkdir cyan
[sedu50@login03 Linux]$ touch my.txt
[sedu50@login03 Linux]$ ls
blue  cyan  green  my.txt  red
[sedu50@login03 Linux]$ mv my.txt ./cyan/
[sedu50@login03 Linux]$ cd cyan/
[sedu50@login03 cyan]$ ls -l
total 0
-rw-r--r-- 1 sedu50 in0163 0 Jan 24 14:56 my.txt
[sedu50@login03 cyan]$ cp my.txt my2.txt
[sedu50@login03 cyan]$ ls -l
total 0
-rw-r--r-- 1 sedu50 in0163 0 Jan 24 14:56 my2.txt
-rw-r--r-- 1 sedu50 in0163 0 Jan 24 14:56 my.txt
[sedu50@login03 cyan]$ mv my.txt ../you.txt
[sedu50@login03 cyan]$ cd ../
[sedu50@login03 Linux]$ ls -l
total 16
drwxr-xr-x 2 sedu50 in0163 4096 Jan 24 14:07 blue
drwxr-xr-x 2 sedu50 in0163 4096 Jan 24 15:00 cyan
drwxr-xr-x 2 sedu50 in0163 4096 Jan 24 14:22 green
drwxr-xr-x 3 sedu50 in0163 4096 Jan 24 14:46 red
-rw-r--r-- 1 sedu50 in0163    0 Jan 24 14:56 you.txt
```

Directory/File Management

➢ rm and mv

```
[sedu50@login03 Linux]$ ls -l
total 4
drwxr-xr-x 2 sedu50 in0163 4096 Jan 24 15:00 cyan
-rw-r--r-- 1 sedu50 in0163    0 Jan 24 15:09 you.txt
[sedu50@login03 Linux]$ rm -i you.txt
rm: remove regular empty file ‘you.txt’? y
[sedu50@login03 Linux]$ ls -l cyan/
total 0
-rw-r--r-- 1 sedu50 in0163 0 Jan 24 14:56 my2.txt
[sedu50@login03 Linux]$ rm -rf cyan/
[sedu50@login03 Linux]$ ls -l
total 0
```

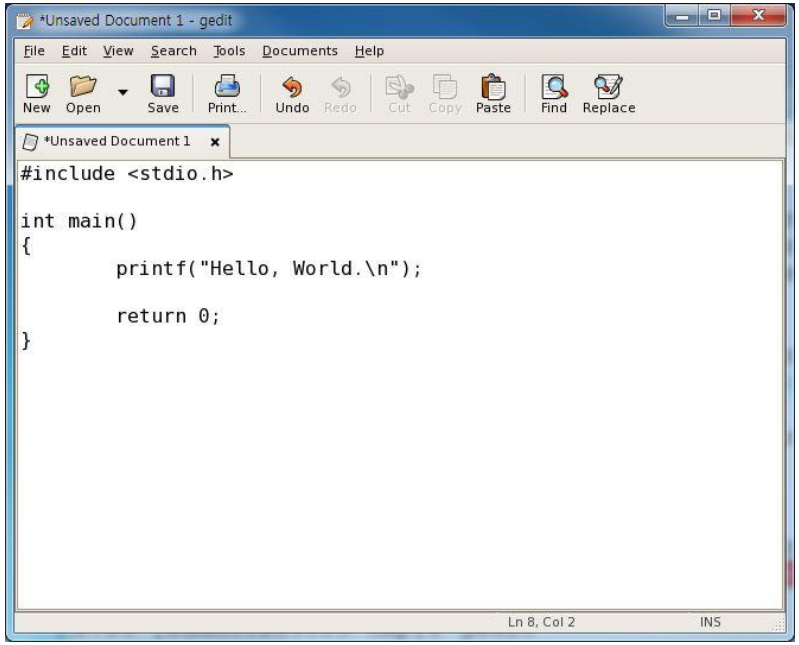
Agenda

- 14:00 ~ 14:40 Linux Overview and Usages
- 14:40 ~ 15:10 vi Editor, sed, and awk Command
- 15:10 ~ 15:30 Break
- 15:30 ~ 16:30 Shell Script Programming
- 16:30 ~ 17:00 Programming Understanding

Editor Types

- vim(vi)
  - The most basic text editor, Included by default in the OS
- nano
  - Easy to use basic text editor
- emacs
  - A powerful and feature-rich editor, Difficult and complicated to use

※ gedit
  - GUI based text editor
  - Easy to use, similar to Notepad



vim Editor

- What is vim editor?
  - Meaning of VIsual display editor
  - Not easy for first timers
  - Installed on most Unix-like systems.
  - Has a significant portion of the functionality of a word processor.
  - Contains a clone of vi called vim.

vim Editor

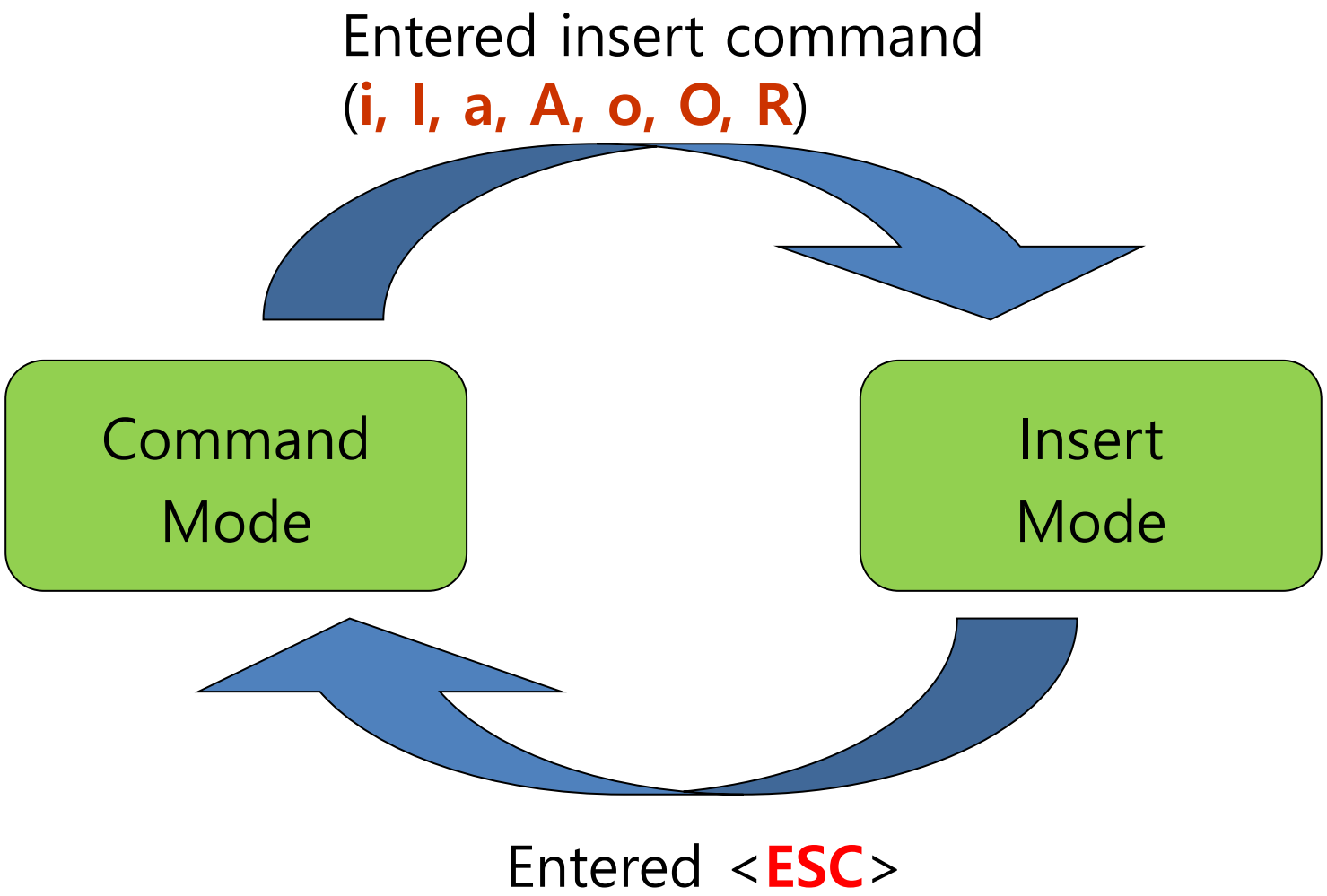
- VI
  - A document editor developed in 1976 by Bill Joy, who developed BSD's C Shell.
  - An abbreviation for 'visual edit', it means writing a document as if writing on paper.
- VIM
  - An abbreviation for VI Improved, which adds several convenient features to the existing vi functions.
  - Extended regular expression syntax, highlighting features, multiple undos, Unicode, etc.

vim Editor

- Open a file
  - $ vi file (Command mode)
  - $ view file (Read mode)

- Modes
  - Insert Mode
    - Everything you type is entered into the editing buffer.
    - When exiting insert mode(When changing to command line mode) : "ESC" key
  - Command Mode
    - Everything you type is interpreted as a command

vim Editor


Entered insert command
(i, I, a, A, o, O, R)
Command Mode
Insert Mode
Entered <ESC>

➢ Move cursor to the top of the screen

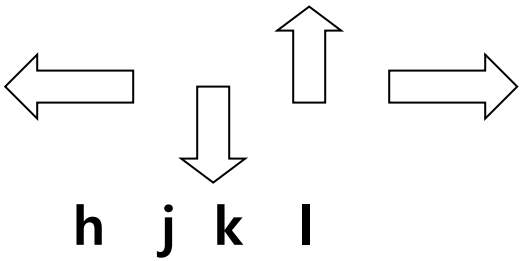
h	Move cursor one character to the left
j	Move the cursor to the same column as the line below
k	Move cursor to the same column as the line above
l	Move cursor one character to the right
<ENTER>	Move cursor to first character of next line
Arrow keys	← ↓ ↑ →

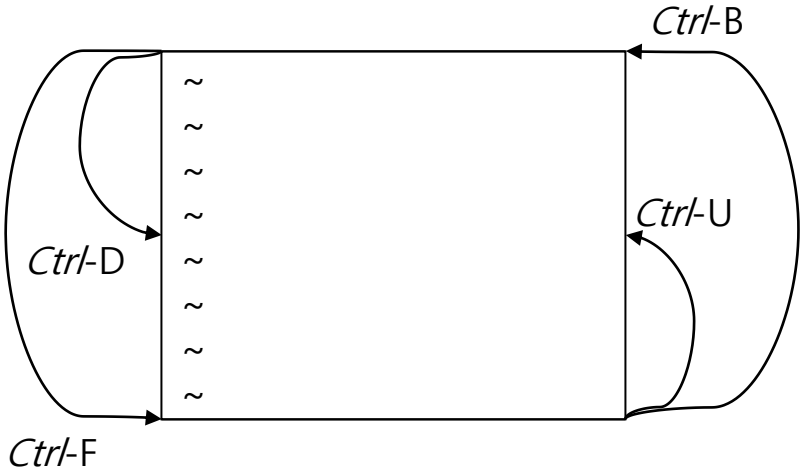
➢ Move the cursor by screen unit

H	Move cursor to the top of the screen
M	Move cursor to middle line of screen
L	Move cursor to bottom line of screen

➢ Cursor Movement

– Go to line nn: nn<Enter>, nnG





➢ Move by word, sentence, or clause

w	Move cursor to first letter of next word
b	Move the cursor to the word in front of where the cursor is located
e	Move cursor to the end of the word where the cursor is located
)	Move cursor to next sentence
}	Move cursor to next section
(	The opposite of)
{	The opposite of }
0	Move cursor to first character of current line
^	Move cursor to the beginning of the current line (Not a tab or space)
$	Move cursor to the end of current line

➢ Move by word, sentence, or clause

^F	Move down one screen
^B	Move up one screen
^D	Move down half a screen
^U	Move up half screen
:$	Go to last line
G	nG : Go to line number n 1G : Move to the first line of the editing buffer G : Go to the last line of the editing buffer

➢ Add, insert, replace, remove data

a	Add data from the right of the current cursor position
i	Add data from the left of the current cursor position
o	Make the line below the current line a blank line
A	Append data to the end of the current line
I	Insert character before the first character of the current line
O	Make the line above the current line a blank line
r	Replace the character where the cursor is currently located
R	Replace characters until the <esc> key is pressed at the current cursor position.
cw	Replace the word at the current cursor position with another word
C	Replace from the current cursor position to the end of the line
s	Replace the character at the current cursor
S	Replace the current cursor line
cc	Replace the line where the cursor is currently located with other content
~	Converts a character at the current cursor position to lowercase or uppercase
<ESC>	Exit data entry mode

➢ Delete data

x	Delete the character where the cursor is currently located
X	Delete the character before the current cursor position
dw	Delete the word to the right of the current cursor position
db	Delete the word to the left of the current cursor position
dd	Delete the line where the cursor is currently located
D	Delete content to the right of where the cursor is currently located
:n1, n2 d	Delete lines n1 ~ n2

➢ Cancel and repeat commands

u	Cancel the command that was performed just before
U	Undo commands performed within a single line
.	Re-execute the command that was performed immediately before

➢ Data movement and copying

yw	Copy the word the cursor is currently located on to the buffer
yy 10yy	Copy current line to buffer Copy 10 lines
nY	Copy n lines from the current cursor position
p	Copy buffer contents to the line to the right or below the current cursor
P	Copy buffer contents to the line to the left or above the current cursor
: n1, n2 y	Copy from line n1 to n2

➢ Combination of lines

J	Connect current line to next line

vim Editor

➢ vi exit

:x ZZ	After saving the file to disk, exit vi.
:wq	After saving the file to disk, exit vi.
:wq [file name]	Save the existing file name to a new file name and exit vi.
:q!	Exit vi without saving the file to disk

➢ Other commands

:set nu	Assign line numbers to files in vi state
:set nonu	Cancel line numbers in a file

➢ vi options

-r [file name]	Recover the specified file
-Wn	When omitted, the window size is set to n
-R	Read-only mode (Prevent overwriting of files)
+ command	Execute the specified ex command before editing
-L	Prints the names of all files
-C command	Start editing by executing a command for a specified editor.
view [file name]	Display the file on the screen in read-only mode in vi
vi file1 file2 file3	Calling 3 files into vi for editing

➢ Search related keys

/Search term	Search for the search term below the row where the cursor is located
?Search term	Search for the search term above the row where the cursor is located
n or /	Moves the cursor position to the next position of the search term
N or ?	Moves the cursor position to the previous location of the search term.

➢ File related commands

:w [file name]	Save changed file contents
:q	Vi exit
:e [file name]	After running Vi, load a specific file and edit it. :e# Reopen a previously edited file
:r [file name]	Load a specific file

➢ Substitution related commands

:s/pattern/replace	Replace in current line
:%s/old/new/g	Replace specific words in a file with new words
:%s/^old/new/g	Replace only words starting with a specific word in a file with a new word
:%s/old$/new/g	Replace only words ending with a specific word in a file with a new word
:%s/aaa//g	Delete the word aaa in the file
:10,50s/old/new/g	Replace specific words between 10 and 50 lines in a file with new words

➢ Execute shell commands in vi environment

– In command mode, type ":!" and enter the desired shell command.

:!ls	**Return to the shell and execute the 'ls' command**
:!pwd	**Show current working directory**
:r !command	**Insert the result of command(command) from the cursor position**

– After executing the command, pressing enter will return you to the vi editor command mode.

➢ Open/edit/save multiple files

– vi file1 file2

– vi *.txt

– :n Go to next file, :N Go to previous file

sed vs. awk Command

- Example

example.txt

```
1, John Doe, 30
2, Jane Smith, 25
3, Emily Davis, 40
```

```
$ sed ‘s/, */,/g’ example.txt | awk –F, ‘&3 > 30 {print $2}’
```

- sed
  - Used for text replacement
  - sed ‘s/, */,/g’ example.txt
    - Replace commas with surrounding spaces (,*) and use only commas (,)
    - s means substitution
    - g means global, Replace all items in each line
- awk
  - Used for data extraction
  - awk –F, ‘&3 > 30 {print $2}’
    - -F sets the field delimiter to ,
    - Instructs to print the second field if the third field is greater than 30.

https://somaz.tistory.com/280

sed Command

- Stream editor that filters and transforms text
- Change the output result without changing the original
- Syntex

```
$ sed 's/Text to find / Text to change /' file name
```

- Option
  - -i[SUFFIX], --in-place[=SUFFIX]
    - edit files in place (makes backup if extension supplied).
    - The default operation mode is to break symbolic and hard links.
    - This can be changed with --follow-symlinks and --copy.
  - -n, --quiet, --silent
    - suppress automatic printing of pattern space
- Operator
  - [range]/p: Output the specified range
  - [range]/d: Delete specified range
  - g: Apply to all lines

https://somaz.tistory.com/280

sed Command

➢ Example

– example.txt

```
Hello World
Hello Somaz
Hello promi
```

– File contents are not changed

```
$ sed ‘s/Hello/Hi/’ example.txt
Hi World
Hi Somaz
Hi promi
```

– File contents changed

```
$ sed –i ‘s/Hello/Hi/’ example.txt
Hi World
Hi Somaz
Hi promi
```

https://somaz.tistory.com/280

awk Command

- A program used to select records from a file and manipulate or datafy the values contained in the selected records.
- Basic format and options of the awk command

```
awk [OPTION...] [awk program] [ARGUMENT...]
OPTION
  -F : Specify field delimiter character
  -f : awk program Specify file path
  -v : Specify a specific variable value to be used in an awk program.
awk program
   Specifies the awk program code that awk will execute if the -f option
   is not used.
ARGUMENT
   Specify input file or variable value
```

```
awk [OPTION...] 'pattern { action }' [ARGUMENT...]
```

https://somaz.tistory.com/280

awk Command

➢ Example

– data.txt

```
1, John, 5000
2, Jane, 5500
3, Doe, 6000
```

– Print except first column

```
$ awk –F, ‘{print $2, $3}’ data.txt
John 5000
Jane 5500
Doe 6000
```

– Calculating the sum of specific fields

```
$ awk '{sum += $3} END { print sum }' data.txt
```

– Output only the records selected by comparing specific field values

```
$ awk ‘$3 >= 5500 { print $2, $3 }’ data.txt
```

– Find the maximum value in a column

```
$ awk 'NR==1 {max=$3} $3>max {max=$3} END {print max}' data.txt
```

https://somaz.tistory.com/280

Agenda

- 14:00 ~ 14:40 Linux Overview and Usages
- 14:40 ~ 15:10 vi Editor, sed, and awk Command
- 15:10 ~ 15:30 Break
- 15:30 ~ 16:30 Shell Script Programming
- 16:30 ~ 17:00 Programming Understanding

Agenda

- 14:00 ~ 14:40 Linux Overview and Usages
- 14:40 ~ 15:10 vi Editor, sed, and awk Command
- 15:10 ~ 15:30 Break
- 15:30 ~ 16:30 Shell Script Programming
- 16:30 ~ 17:00 Programming Understanding

Programming Classification Based on Script

- No-Script
  - Compiled to be executable and operates independently.

    ex) Fortran, C, C++
- Script
  - A programming language that runs embedded in other applications.

    ex) Shell, Python

Reference: namu.wiki

Shell Script

- Shell script programming
  - Used when you want to repeatedly use basic commands of Unix/Linux systems.
  - A shell script is a script that operates like a program in order to sequentially use the functions provided by the shell.
  - Operators and options for shell can be used as needed.

- Linux Main Shell
  - bash(Bourne-Again Shell)
  - ksh(Korn Shell)
  - csh(C Shell)

- Check Shell
  - echo $SHELL
  - cat /etc/shells

Startup Script in Linux

- ksh
  - /etc/profile => /etc/profile.d/*.ksh
  - ${HOME}/.profile
  - ${HOME}/.kshrc
  - ref : http://www.kornshell.org
- csh
  - /etc/profile => /etc/profile.d/*.csh
  - ${HOME}/.login
  - ${HOME}/.cshrc
  - ref : https://en.wikipedia.org/wiki/C_shell
- bash
  - /etc/profile => /etc/profile.d/*.sh
  - ${HOME}/.bash_profile
  - ${HOME}/.bashrc

Startup Script in Linux

- Shell script
  - Bundles together system calls, various programming tools, utilities, executables, etc.
  - Access to all kinds of Unix/Linux commands, utilities, and tools
  - Ability to use internal shell commands such as branch statements or loop statements

- Execute permission
  - To run a shell script file, you must have read/execute permissions.
  - If you do not have execution permission but have read permission, you can execute it using the sh command, such as '**sh ./filename.sh**'
  - If you do not have read permission, the script content cannot be read and thus cannot be executed.

Scripting

- Shell script structure
  - Specify the shell to be executed on the first line (#!/bin/bash etc.)
  - Configure the commands to be executed in the middle section
  - Write 'exit 0' to terminate the program(Optional)

- Example
  - vi : vi hello.sh
  - ※ gedit : gedit hello.sh

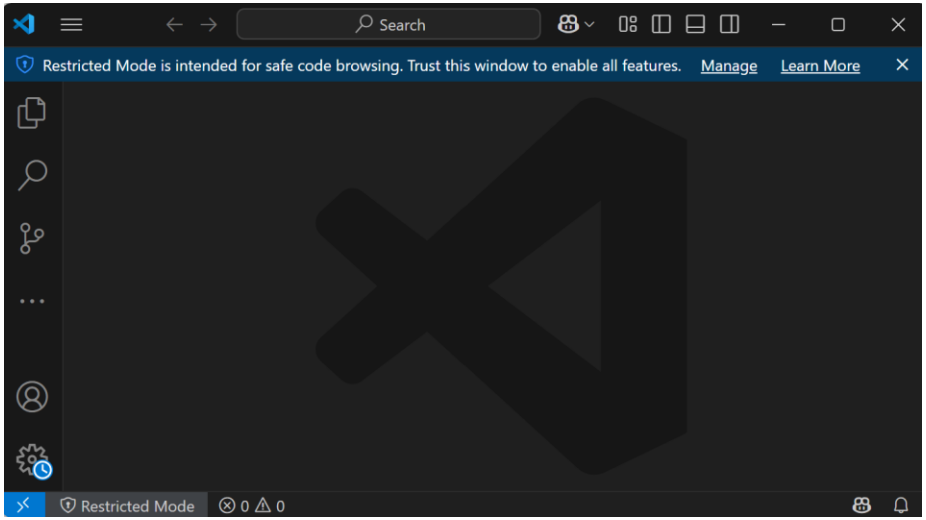
Example

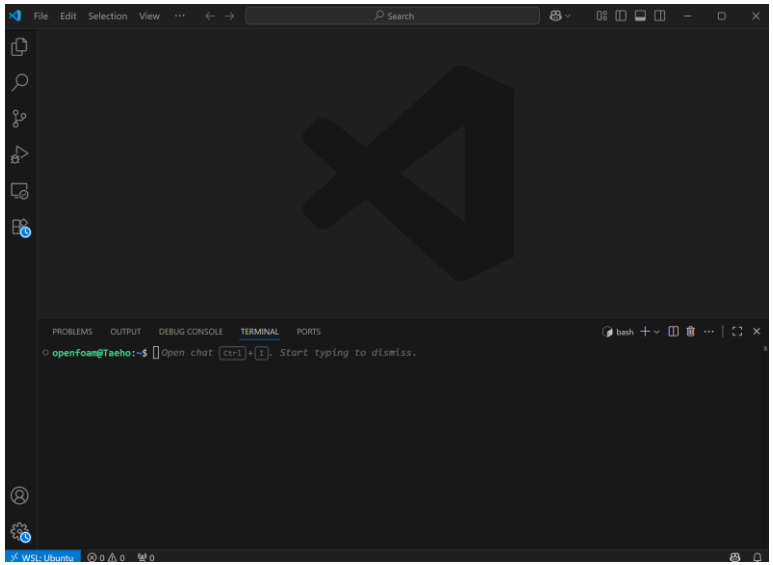
```
#!/bin/bash

echo "Hello"

exit 0
```

- Run after changing permissions
  - chmod u+x hello.sh
  - ./hello.sh
  - sh ./hello.sh

Visual Studio Code in Window Platform - WSL2

- Visual Studio Code install
  - download: https://code.visualstudio.com/download
  - Install the download files
  - Enter the code in the Windows platform search box and press Enter key
- WSL2 connection
  - Select the WSL that is already installed by clicking the connection indicator icon at the bottom left.





➢ Comment out

- Lines starting with # are commented out
- If it comes in the first column, the entire line is a comment
- In the case of '#' following a command, the right side of # is a comment.
- #! indicates the execution shell.

Example

```
#!/bin/bash

#print hello
echo "hello" #'$?' is 0 or not 0
echo $?

exit 0
```

➢ Command separator

- Separate commands using semicolon(;).
- Used when you want to use more than two commands on one line.
- However, double semicolons (;;) end the case statement.

Example

```
#!/bin/bash

echo hello; echo $?

echo hello
echo $?  #'$?' is 0 or not 0

exit 0
```

Variables

- Shell script variables
  - If the variable name is variable, the variable value is expressed as $variable.
    - $variable is a short form of ${varivable}
  - Referencing the value of a variable is called variable substitution
  - The initial variable type is not necessarily determined
  - The variable type is determined based on the type of the value when assigned.
  - It is recommended not to perform assignments or operations between variables of different types.
  - Variable names only allow letters (a-z, A-Z), numbers (0-9), and underscores (_).
  - When referencing declared variable values, the results differ depending on quoting(' ', " ").

Variables

- assignment; =
  - Initialize or change variable values
  - In Shell, it performs both the role of assigning arithmetic expressions and strings.
  - There must be no space before the assignment operator.
  - Different meaning from comparison operators used in test statements
  - When assigning a value to a variable, use the ‘=’ operator.
  - No spaces allowed between “="

    ex) variable = 0 (x)

    variable=0 (o)

Variables

Example 1	Example 2

#!/bin/bash
 	
# print hello 	
variable1=world
echo hello $variable1
 	
variable2=$?
echo $variable2 # '$?' is 0
 	
exit 0
	#!/bin/bash
	
	hello="A B C D" # variable assignment
	echo $hello #
	echo "$hello" # double quot
	echo '$hello' # single quot
	
	exit 0

$ sh ./var_test.sh $ ./var_test.sh	$ sh ./var_test2.sh $ ./var_test2.sh

Variables

Example 3	Example 4
#!/bin/bash var_a=33 echo "a = $var_a" var_b=$var_a echo "b = $var_b" var_a=`echo hello` echo $var_a var_b='echo hello' echo $var_b var_a=`ls -al` echo $var_a exit 0	#!/bin/bash echo "File Name: $0" echo "First Parameter : $1" echo "Second Parameter : $2" echo "Quoted Values: $@" echo "Quoted Values: $*" echo "Total Number of Parameters : $#" exit 0
$ sh ./assignment.sh $ ./assignment.sh	$ sh ./arg.sh Hello World $ ./arg.sh Hello World

Special Variables

➢ Unix Variable(reserved variables)

– Special characters are assigned to perform specific functions in Unix/Linux.

Variable	Description
$0	The file name of the current file script.
$n	Command line arguments
$#	Number of arguments used in the script
$*	Combine all arguments used in the script into one string.
$@	All arguments used in the script are grouped into a list.
$?	Stores the return value of the last executed command (0 : normal execution, others : error msg)
$$	The ID value of the process running in the current shell.
$!	The ID value of the process running in the background in the current shell.

Special Variables

- Exit and Exit status ($?)
  - If the command execution is successful, 0 is stored in $?
  - If the command execution fails, the exit status value is stored in $?
  - Useful when you want to perform different actions depending on whether the command execution was successful or not.

Example

```
#!/bin/bash

echo hello
echo $?    # The status value 0 is printed because the command worked properly.

asdasdasd          # Unknown command
echo $?              # Outputs results for non-zero error conditions

exit 0
```

Quoting

- Enclose any string in quotes(“ ”, ‘ ’, ` `)
- Prevents certain characters (special characters) from expanding to results other than intended
- “ ”(double quotation)
  - Used when referencing a general string or variable
  - Preserve all special characters except $(dollar), `(back quotation) , ₩(escape, back slash)
- ‘ ’ (single quotation)
  - Print all characters literally except ` `(back quotation)
  - Variable references do not occur because the reference semantics of $ is not allowed.
  - ₩ is printed literally
  - A slightly more rigorous method than double quotation
- ` `(back quotation)
  - Command substitution
  - It means to execute a string declared with back quotation as a shell command.

Operators

➢ Arithmetic Operator

Operator	Description	Operator	Description
+	Plus	-	Minus
*	Times	/	Slash(Division)
**	Exponent	%	Modulo, Mod
+=	Plus Equal	-=	Minus Equal
*=	Times Equal	/=	Slash Equal
%=	Modulo Equal	=	Assignment

➢ Logical Operator

Operator	Description	Operator	Description
&&	and	||	or
==	equality	!=	Not Equality

Operators

➢ Relational Operator

Symbol	Description	Operator	Description
-eq	Equality	-gt	Greater Than
-ne	Not Equality	-lt	Less Than
-ge	Greater than or Equal	-le	Less than or Equal

➢ File Test Operator

Operator	Description	Operator	Description
-b	Block special file	-p	Pipe file
-c	Character special file	-t	File descriptor is associated with a terminal device
-d	Directory file	-u	user ID(SUID) bit is set
-f	Regular files (not directories or links)	-r	Read permission
-g	group ID(SGID) bit is set	-w	Write permission
-k	sticky bit is set	-x	Execute permission
-s	file size > 0	-e	Existing file

Arithmetic & Logical Operator Examples

Example - expr	Example - let

#!/bin/bash
a=10 ; b=20
 	
val=`expr $a + $b` ; echo "a + b : $val"
val=`expr $a - $b` ; echo "a - b : $val"
val=`expr $a ₩* $b` #### * : meta character
echo "a * b : $val"
val=`expr $b / $a` ; echo "b / a : $val"
val=`expr $b % $a` ; echo "b % a : $val"
if [$a == $b] ; then
  echo "a is equal to b"
fi
if [$a != $b] ; then
  echo "a is not equal to b"
fi
	#!/bin/bash
	a=10 ; b=20
	let val=a+b ; echo "a + b : $val"
	let val="a - b" ; echo "a - b : $val"
	let val='a * b' ; echo "a * b : $val"
	let val=b/a ; echo "b / a : $val"
	let val=b%a ; echo "b % a : $val"
	if [$a == $b]
	then
	  echo "a is equal to b"
	fi
	if [$a != $b]
	then
	  echo "a is not equal to b"
	fi

$ sh ./op1.sh $ ./op1.sh	$ sh ./op2.sh $ ./op2.sh

Control Structure – if Statement

- if Statement
  - If the expression in if or elif is true, then execute the then statement.
  - In other cases, execute the following conditional statement:
  - End with fi

Syntax

```
if [ expression 1 ]
then
   Statement(s) to be executed if expression 1 is true
elif [ expression 2 ]
then
   Statement(s) to be executed if expression 2 is true
elif [ expression 3 ]
then
   Statement(s) to be executed if expression 3 is true
else
   Statement(s) to be executed if no expression is true
fi
```

Control Structure – if Statement

Example 1	Example 2

#!/bin/bash
a=10
b=20
if [$a == $b]
then
  echo "a is equal to b"
elif [$a -gt $b]
then
  echo "a is greater than b"
elif [$a -lt $b]
then
  echo "a is less than b"
else
  echo "None of the condition met"
fi
	#!/bin/bash
	if [-e readme.txt] ; then
	  echo 'The file "readme.txt" exists.'
	else
	  echo 'The file "readme.txt" does not exist.'
	fi

$ sh ./cond_if1.sh $ ./cond_if1.sh	$ sh ./cond_if2.sh $ ./cond_if2.sh

Control Structure – CASE Statement

➢ CASE

- Unlike IF, which is a sequential branch statement, it is a parallel branch form.
- In some cases, branching statements can be handled more effectively than IF statements.

  (all of the branches depend on the value of a single value)
- End with esac

Syntax

```
case word in
  pattern1)
     Statement(s) to be executed if pattern1 matches
     ;;
  pattern2)
     Statement(s) to be executed if pattern2 matches
     ;;
  pattern3)
     Statement(s) to be executed if pattern3 matches
     ;;
esac
```

Control Structure – CASE Statement

Example 1	Example 2

#!/bin/bash
 	
echo -n “Input String”
read FRUIT
 	
case "$FRUIT" in
  "apple") echo "Apple pie is quite tasty."
  ;;
  "banana") echo "I like banana nut bread."
  ;;
  "kiwi") echo "New Zealand is famous for kiwi."
  ;;
esac
	#!/bin/bash
	option="${1}"
	case ${option} in
	  -f) FILE="${2}"
	  echo "File name is $FILE"
	  ;;
	  -d) DIR="${2}"
	  echo "Dir name is $DIR"
	  ;;
	  *)
	  echo "`basename ${0}`:usage: [-f file] | [-d directory]"
	  exit 1 # Command to come out of the program with status 1
	  ;;
	esac

$ sh ./cond_case1.sh $ ./cond_case1.sh	$ sh cond_case2.sh -d readme.txt $ ./cond_case2.sh -d readme.txt

LOOP Type

- Code commands that can efficiently process repetitive commands

- Types of Loops

while

```
while command
do
   Statement(s)
done
```

until

```
until command
do
   Statement(s)
done
```

for

```
for var in word1 word2 ...
do
   Statement(s)
done
```

select

```
select var in word1 word2 ...
do
   Statement(s)
done
```

LOOP Example

Example - while	Example - until

#!/bin/bash
 	
a=0
 	
while [$a -lt 10]
do
  echo $a
  a=`expr $a + 1`
done
	#!/bin/bash
	
	a=0
	
	until [**!** $a -lt 10]
	do
	  echo $a
	  a=`expr $a + 1`
	done

$ sh ./loop_while.sh $ ./loop_while.sh	$ sh ./loop_until.sh $ ./loop_until.sh

LOOP Example

Example - for	Example - select

#!/bin/bash
 	
for var in 0 1 2 3 4 5 6 7 8 9
do
  echo $var
done
	#!/bin/bash
	
	select DRINK in tea cofee water juice appe all none
	do
	  case $DRINK in tea|cofee|water|all)
	  echo "Go to canteen"
	  ;;
	  juice|appe)
	  echo "Available at home"
	  ;;
	  none)
	  break
	  ;;
	  *) echo "ERROR: Invalid selection"
	  ;;
	  esac
	done

$ sh ./loop_for.sh $ ./loop_for.sh	$ sh ./loop_select.sh $ ./loop_select.sh

Numerical Constants

- How to notate numeric constants
  - Decimal : 64
  - Octal(start with 0) : 0100
  - Hexadecimal (start with 0x) : 0x40
  - other positional numeral systems (Base # Number, Base is 2~32) : 32#20

Example

```
#!/bin/bash

let "a=64"
let "b=0100"
let "c=0X40"
let "d=32#20"

echo $a $b $c $d                    # 64 64 64 64

exit 0
```

Float Calculation

- Impossible real number operation
  - Specific shells do not support floating point operations(ksh, zsh support)
  - Recommended use of other scripting languages : AWK, Perl, python, etc.
  - Bypass possible using Linux's pipe function(bc, AWK, etc.)

Example

```
#!/bin/bash

# Output real value
a=`echo 1234.56 | bc`                # How to store real values in script variables
echo $a
printf "%f₩n" 1234.56                # c function, Direct operation is not possible

# Outputting results using PIPE
echo 1234.56+1 | bc
echo 1234.56+1 | awk '{printf "%f₩n", $1}'
a=`echo 1234.56 + 1 | bc` ; echo $a
exit
```

➢ Example (including awk and sed command)

– Three data files containing an individual's name, age, and score (data1.csv, data2.csv, data3.csv)

data1.csv

Name	Age	Score
Alice	25	88
Bob	30	92
Charlie	22	70

data2.csv

Name	Age	Score
David	28	85
Eve	35	95

data3.csv

Name	Age	Score
Frank	40	60
Grace	27	82

– Programming guide
  - Count score greater than or equal to 80.
  - Output the average score and the number of people for each data.
  - Output the total average score and the total number of people.

➢ Practice

```
#!/bin/bash

TOTAL_SUM=0
TOTAL_COUNT=0

echo "=== ASEAN Data Analysis ==="

for file in data*.csv
do
    echo ">>> Loading: $file"
    cleaned=$(sed                    )
    result=$(echo "$cleaned" | awk -F',' '
    {
        name=$1
        score=$3

        if (          {
                                        
            sum += score
                                        
        }
    }
    END {
        if (count > 0) {
            print "FILE_AGE_SUM", sum_age
            print "FILE_SUM", sum
            print "FILE_COUNT", count-1
        }
    }')

    echo "$result"

    file_sum=$(echo "$result" |                          )
    file_count=$(echo "$result" |                          )

    TOTAL_SUM=$((TOTAL_SUM + file_sum))
    TOTAL_COUNT=$((TOTAL_COUNT + file_count))

    echo "----------------------------"
done

echo "=== Total result ==="

if [ "$TOTAL_COUNT" -gt 0 ]; then
    echo "TOTAL_COUNT: $TOTAL_COUNT"
    echo "TOTAL_SUM: $TOTAL_SUM"
    echo "AVERAGE:                                    "
else
    echo "No data"
Fi
```

Agenda

- 14:00 ~ 14:40 Linux Overview and Usages
- 14:40 ~ 15:10 vi Editor, sed, and awk Command
- 15:10 ~ 15:30 Break
- 15:30 ~ 16:30 Shell Script Programming
- 16:30 ~ 17:00 Programming Understanding

Classification of Programming language

- A language is designed to give commands or calculations to a machine(computer) and to enable communication with the machine.
- Classification by level:
  - Low-level languages: Easy for computers to understand because it has little abstraction

    ex) Assembly language, Machine language
  - High-level languages: A highly abstract language, Easy to understand, but requires complex translation for computers to understand

    ex) Fortran, C/C++, Python, etc.

Reference: namu.wiki

Classification of Programming language

- Classification by Analysis method:
  - AOT(Ahead Of Time) compile language:
    - Translate the source code into machine language in advance and run it

      ex) C/C++, Rust, Pascal, Haskell
  - JIT(Just-In-Time) compile language:
    - The first compiler generates bytecode. → A virtual machine (VM) that executes bytecode, Most languages apply

      ex) Java, JavaScript, C#, JVM language
  - Interpreter compile language:
    - Read the source code line by line, translate it as needed, and execute it.

      ex) Python, Basic, Shell, PHP

Reference: namu.wiki

Classification of Programming language

- Programming Paradigm:
  - Imperative Language

    A programming language that expresses how a program performs a task.

    Based on Turing machine

    - Procedural Language
      - A method of solving a problem by dividing it into units of procedures.

        ex) C, Pascal, Fortran
    - Object-oriented language
      - The goal is to create objects by bundling data and methods to process that data, and to assemble objects.

        ex) Java, C++

Reference: namu.wiki

Classification of Programming language

- Programming Paradigm:
  - Declarative language

    Concepts that contrast with imperative language
    - Functional programming language
      - Based on lambda algebra
    - Logic programming language
      - A general term for programming languages that use the first-order logic of mathematical logic as a model.

Reference: namu.wiki

High-level Programming Language

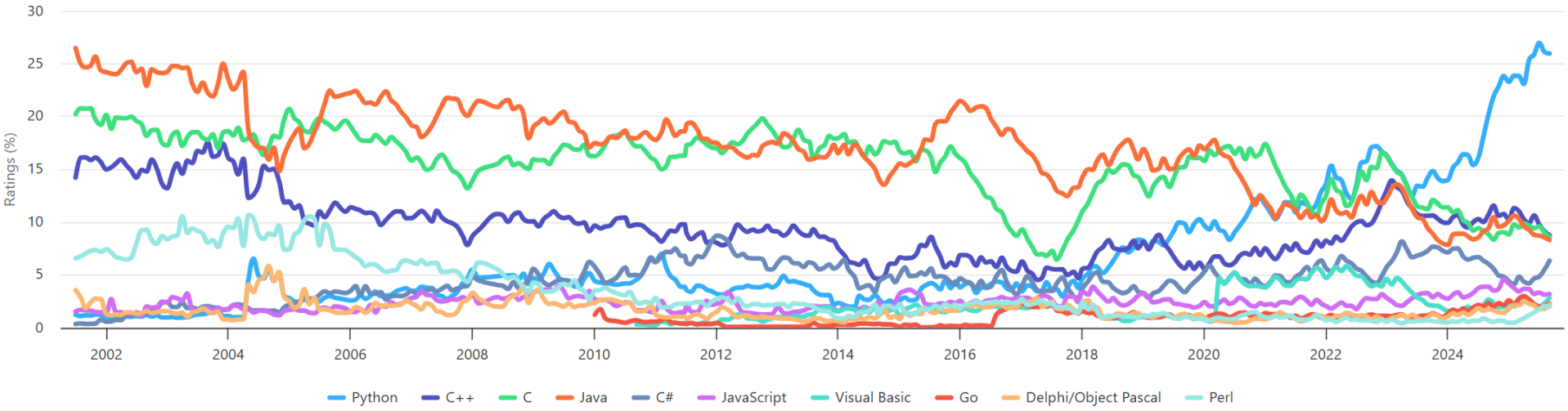
- The TIOBE Programming Community index(Sep. 2025)
  - An indicator of the popularity of programming languages

Sep 2025	Sep 2024	Change	Programming Language	Ratings	Change
1	1		Python	25.98%	+5.81%
2	2		C++	8.80%	-1.94%
3	4	^	C	8.65%	-0.24%
4	3	v	Java	8.35%	-1.09%
5	5		C#	6.38%	+0.30%
6	6		JavaScript	3.22%	-0.70%
7	7		Visual Basic	2.84%	+0.14%
8	8		Go	2.32%	-0.03%
9	11	^	Delphi/Object Pascal	2.26%	+0.49%
10	27	^^	Perl	2.03%	+1.33%
11	9	v	SQL	1.86%	-0.08%
12	10	v	Fortran	1.49%	-0.29%

Source: https://www.tiobe.com/tiobe-index/

High-level Programming Language

➢ The TIOBE Programming Community index



Source: https://www.tiobe.com/tiobe-index/

Programming languages test: Hello World!

- Fortran

```
program hello
  print *, 'Hello World!'
end program hello
```

```
gfortran -o hello_world hello_world.f90
(ifort -o hello_world hello_world.f90)
./hello_world
```

- C

```
#include <stdio.h>
int main(){
    printf("Hello World!");
    return 0;
}
```

```
gcc -o hello_world hello_world.c
(icc -o hello_world hello_world.c)
./hello_world
```

- C++

```
#include <iostream>
using namespace std;
int main(){
    cout<<"Hello World!";
    return 0;
}
```

```
g++ -o hello_world hello_world.cpp
(ipc –o hello_world hello_world.cpp)
(icpc -o hello_world hello_world.cpp)
./hello_world
```

- Python

```
print('Hello World!')
```

```
python hello_world.py
```

Fortran

- FORTRAN(FORmula TRANslation):

  Refers to the ability of the language to translate mathematical formulas and expressions into a form that can be processed by a computer

  https://fortran-lang.org/

- Developed by IBM in the early 1950s
- The oldest and first high-level language
- It had a huge influence on the birth of high-level languages such as the C language.
- Interoperability
  - Commonly used regardless of computer model
- Easy and simple grammar and commands
  - Available for short term use
  - All commands are expressed in sentences
  - A sentence consists of executable and non-executable statements.

Fortran Example

- Cylinder volumn calculation

```
program arithmetic
 implicit none
 real :: pi
 real :: radius
 real :: height
 real :: area
 real :: volume
 pi = 3.1415927
 print *, 'Enter cylinder base radius:'
 read(*,*) radius
 print *, 'Enter cylinder height:'
 read(*,*) height
 area = pi * radius**2
 volume = area * height
 print *, 'Cylinder radius is: ', radius
 print *, 'Cylinder height is: ', height
 print *, 'Cylinder base area is: ', area
 print *, 'Cylinder volume is: ', volume
end program arithmetic
```

C

- A system description language developed by Dennis Richie, a systems programmer at Bell Laboratories in the United States in 1972.
- More than 90% of real UNIX operating systems are written on it.
- Currently, it is widely used due to its diverse operators, data types, and function libraries, and its high portability.
- Write a program by function

C Example

➢ Pi calculation example

```
#include <time.h>
#include <stdio.h>
static long num_steps=1000000000;
double step;

int main()
{
   int i; double x, pi, sum=0.0, t1, t2;
   step =1.0/(double)num_steps;
   t1 = (double)clock() /(double) CLOCKS_PER_SEC;
   for(i=0;i<num_steps;i++){
      x=(i+0.5)*step;
      sum+=4.0/(1.0+x*x);
   }
   pi = step*sum;t2 = (double)clock() /(double)CLOCKS_PER_SEC;
   printf("pi = %17.15lf₩n",pi);
   printf("elapsed time = %f₩n",t2-t1);
}
```

C++

- Announced in 1985 by Bjarne Stroustrup of AT&T Bell Labs based on the C language.
- https://isocpp.org/
- Added content to support object-oriented programming in the C language.
  - Abstraction
    - Use class to manage complex and abstract variables or functions
  - Data hiding
    - Specifying a private attribute to protect functions or variables from outside the class
  - Encapsulation
    - Capsule-style organization by dividing into status and function
  - Inheritance
    - Create a simple class that adds functionality by leveraging other classes
  - Polymorphism
    - Functions with the same name can be defined multiple times (if the data types or number of arguments are different)

C++ Example

test.C

```
#include <iostream>
using namespace std;
int main()
{
double d1=2.1;
double d2=3.7;
double* d3; //d3 is a pointer, currently not pointing at anything
d3 = &d1; //Now d3 points at the memory location of d1
cout << "d1: " << d1 << endl;
cout << "d2: " << d2 << endl;
cout << "d3: " << d3 << endl;
cout << "*d3: " << *d3 << endl;
d3 = &d2; //Now d3 points at the memory location of d2
cout << "d3: " << d3 << endl;
cout << "*d3: " << *d3 << endl;
}
```

```
>> g++ test.C -o test3
```

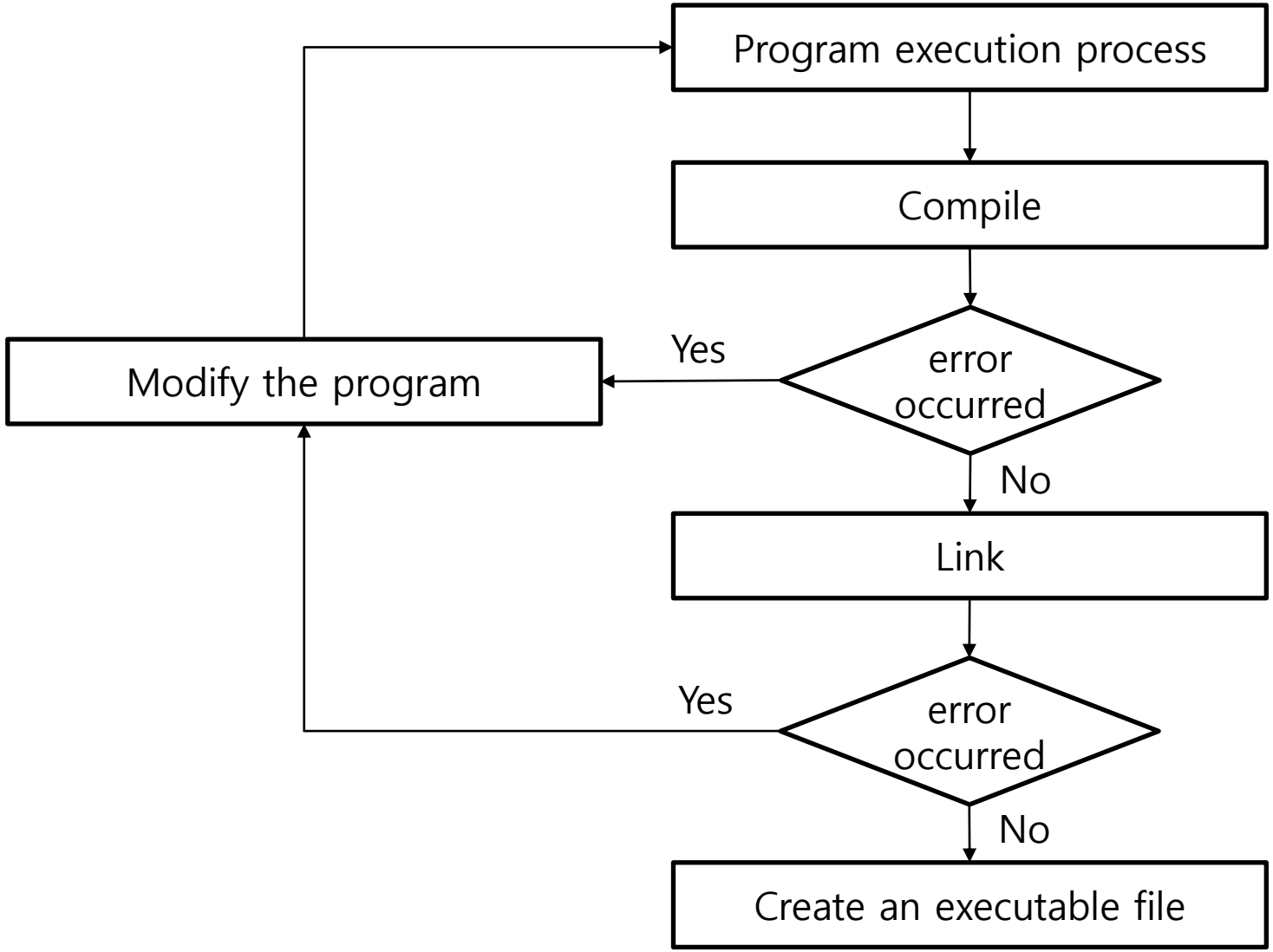
C++ Example

- Class example of creating an object and calling a member function
  - Point: Designed data type
  - startPt, endPt: Practical using object

```
#include <iostream>
using namespace std;
#include "point.H"

int main()
{
    Point startPt, endPt;
    startPt.setPoint(10, 10);
    endPt.setPoint(50, 50);

    startPt.prnPoint();
    endPt.prnPoint();
    return 0;
}
```

```
class Point {
    int x, y;

    public:
        void setPoint(int a, int b)
        {
            x = a;
            y = b;
        }

        void prnPoint() {
            cout << "(" << x << ", " << y << ")";
        }
};
```

Program execution process



Program execution process

- Compile: The process of converting source code written by a developer into binary code
  - Object file: File converted through compilation
  - Object file contains machine code, symbol table, debug information, and relocation information.
  - ELF(Excutable and Linking Format) file
    - ELF header-program header table-section header table
- Link: Process of creating an executable file
  - Linker: A program that takes one or more object files and merges them into a single executable file

Assembly(Assembler) language

- Assembler: A computer language translation program that interprets assembler language into machine language object code.
- Assembly(Assembler) language:
  - A low-level computer programming language that corresponds one-to-one with machine language
  - By setting mnemonic symbols that are easy for people to understand, people can control the computer's behavior more easily.
- Example
  - 0110000 01100001

    (Machine language instructions for x86 series CPUs)

    → mov al, 061h (Assembly language)

Hexadecimal 61

One of the registers that stores variables in the CPU

Mnemonic modified from the word 'move'

Reference: wikipedia

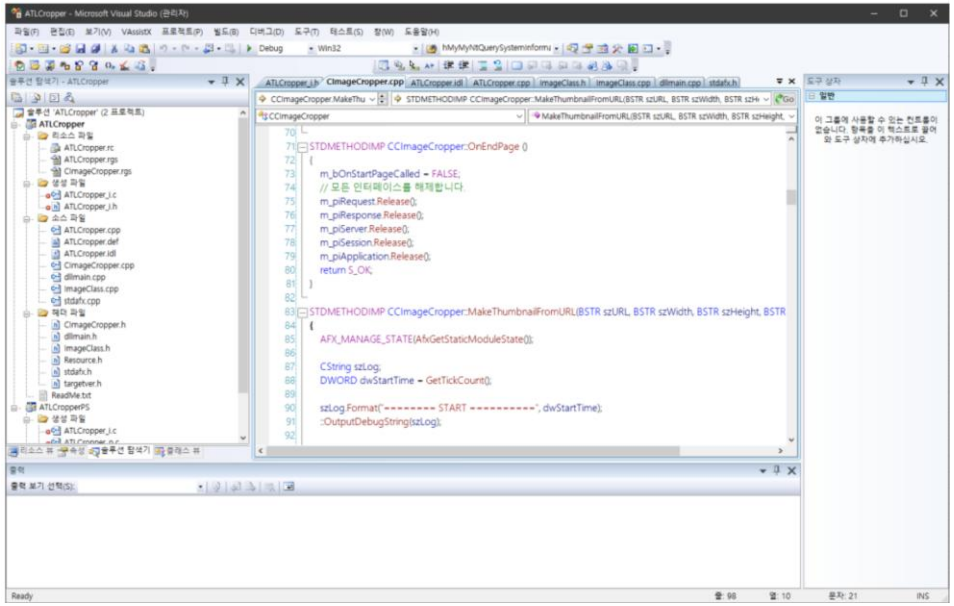
Example of Assembly program ‘Hello World!’

```
adosseg
.model small
.stack 100h

.data hello_message db 'Hello, World!',0dh,0ah,'$'

.code main proc
        mov ax, @data
        mov ds, ax

        mov ah, 9
        mov dx, offset hello_message
        int 21h

        mov ax, 4C00h
        int 21h

main endp
end main
```

Reference: wikipedia

Debugger(or debugging tool)

- A computer program used to test and debug other target programs.
- List of debuggers:
  - GDB: Basic debugger for the GNU software system.
  - Visual Studio: Microsoft's flagship IDE, Optimized for Windows platform



Reference: wikipedia

Compile

- Meaning of compile
  - The process of converting user-written source code into binaries so that the computer can execute it.
  - In a broad sense, it includes including libraries that have already been written.

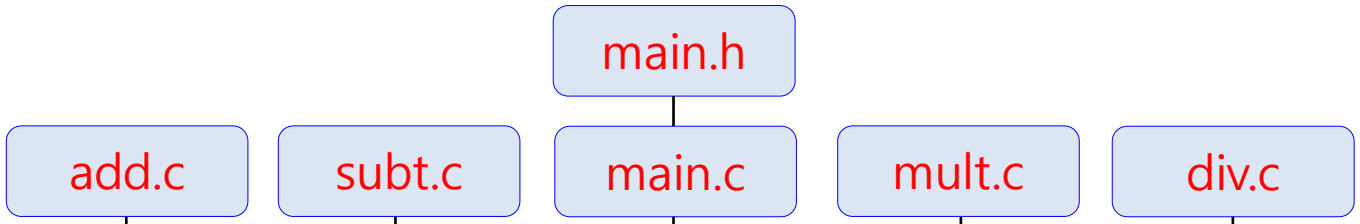
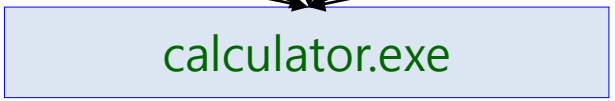
- Compiler
  - Role that performs compilation tasks
  - It is standardized according to the type of language used, and there are several versions depending on the manufacturer.

- Representative C compilers
  - icc(Intel), xlc(IBM), gcc(GNU), cc(Cray)

- Representative Fortran compilers
  - ifort(Intel), xlf(IBM), gfortran(GNU), ftn(Cray)

Compiling, Linking, Execution file for Multi-code

➢ example : calculator program

program source file	
program objective file	
program executable file	

	$ gcc -c add.c -o add.o $ gcc -c subt.c -o subt.o
	$ gcc -c mult.c -o mult.o $ gcc -c div.c -o div.o
	$ gcc -c main.h main.c -o main.o
	$ gcc add.o subt.o mult.o div.o main.o -o calculator.exe
	$ ./calculator.exe

Compiling, Linking, Execution File for Multi-code

- make(=gmake)
  - Write a Makefile
  - Excute 'make'

```
OBJECTS = add.o subt.o mult.o div.o main.o
EXECUTABLE = calculator.exe

$(EXECUTABLE) : $(OBJECTS)
    gcc $(OBJECTS) -o $(EXECUTABLE)

add.o : add.c
    gcc -c add.c -o add.o

subt.o : subt.c
    gcc -c subt.c -o subt.o

mult.o : mult.c
    gcc -c mult.c -o mult.o

div.o : div.c
    gcc -c div.c -o div.o

main.o : main.h main.c
    gcc -c main.h main.c
```

} Macro declaration

} Use macro

- cmake
  - Write a CMakeLists.txt
  - Makefile automatic generation
  - Excute 'make'

CMake
Cross-platform Make

```
cmake_minimum_required(VERSION 3.0.1)
project(example)
add_executable(${PROJECT_NAME} main.cpp)
```

Python

- Created by Guido van Rossum in 1992
- Used in more than half of software made by Google
- Used on well-known platforms such as Dropbox and Instagram
- Easy to understand code, easy to collaborate and maintain
- Interpreter language
- Strict about line breaks (Tab) in terms of grammatical structure – called Pythonic code
- The code is intuitive and easy to understand.

➢ Simple Example

```
from math import *
print(pow(3,2))
print(sqrt(3))

L = [“A",“B",“C"]
T = (1, 2, 3)
S = “TAEHOYOON"
total = 0
for i in L:
    print(i)
for i in T:
    print(i)
for i in S:
    print(i)
for i in (1,10):
    total += i
print (total)
```

PI Calculation

```
# Initialize denominator
k = 1
 # Initialize sum
s = 0
for i in range(1000000):
    # even index elements are positive
    if i % 2 == 0:
        s + = 4/k
    else:

    # odd index elements are negative
        s -= 4/k
     # denominator is odd
    k += 2

print(s)
```

Q&A

Thank
you!